

The Definitive Guide to NetSuite SuiteCloud Development: Architecture, Configuration, and Deployment

Executive Summary

The enterprise resource planning (ERP) landscape has evolved significantly over the last decade, transitioning from rigid, monolithic systems to flexible, cloud-native platforms. NetSuite, as a pioneer in cloud ERP, has facilitated this shift through the SuiteCloud Platform, a robust ecosystem that allows for extensive customization and application development. At the heart of this ecosystem lies the SuiteCloud Development Framework (SDF), a file-based development environment that enables developers to manage NetSuite customizations as code. This paradigm shift—moving from browser-based, point-and-click changes to a modern Integrated Development Environment (IDE) workflow—aligns NetSuite development with industry-standard software engineering practices, including version control, continuous integration, and automated testing.

This comprehensive research report serves as an exhaustive manual for the SuiteCloud Development Framework, specifically tailored for the Visual Studio Code (VS Code) environment. It addresses the critical requirements of modern NetSuite development, including the configuration of the SuiteCloud extension, the intricate management of authentication credentials (tokens, Application IDs, and Account IDs), and the strategic differentiation between Account Customization Projects (ACP) and SuiteApp projects. Furthermore, this report provides a granular analysis of the deployment lifecycle for SuiteScript 2.x, detailing the XML metadata structures required to bridge the gap between local JavaScript files and server-side NetSuite records.

As of 2024 and heading into 2025, the platform is undergoing a mandatory evolution in authentication protocols, deprecating legacy Token-Based Authentication (TBA) in favor of robust OAuth 2.0 standards. This report rigorously examines this transition, providing the technical depth necessary for architects and developers to navigate the change without disrupting critical business operations. By synthesizing technical documentation, best practices, and architectural patterns, this document aims to elevate the reader from functional proficiency to expert mastery of the SuiteCloud ecosystem.

1. The SuiteCloud Environment: Architecture and Prerequisites

The foundation of any successful software development lifecycle (SDLC) is a correctly configured local environment. Unlike the historical approach of editing scripts directly within the NetSuite browser UI, SDF requires a specific local toolchain to compile, validate, and synchronize code with the NetSuite backend. This section details the architectural dependencies and installation procedures required to establish a functioning SuiteCloud environment in Visual Studio Code.

1.1 The Dependency Chain: JDK and Node.js

The SuiteCloud Extension for VS Code is not a standalone binary; it functions as a sophisticated wrapper around the SuiteCloud Command Line Interface (CLI), which in turn relies on a chain of system-level dependencies to execute operations. Understanding this dependency chain is crucial for troubleshooting connectivity and compilation errors.

1.1.1 Java Development Kit (JDK) Configuration

The core communication layer of the SuiteCloud CLI—responsible for the heavy lifting of SOAP and REST web services calls to the NetSuite backend—is built on Java. Consequently, the environment strictly requires a 64-bit installation of the **Oracle Java SE Development Kit (JDK)**.

- **Version Specificity:** The ecosystem has moved beyond older Java versions. The current mandatory requirement is **Oracle JDK 17** or **Oracle JDK 21**.¹ Using legacy versions, such as JDK 11 or JDK 8, or non-Long Term Support (LTS) versions between 17 and 21, will result in the CLI failing to initialize or throwing obscure "class not found" errors during authentication handshakes.
- **Architecture:** It is imperative to use the 64-bit version of the JDK. NetSuite's data processing involves large XML payloads during deployment validation, and 32-bit Java Virtual Machines (JVMs) often hit memory ceilings that cause deployment crashes.¹
- **Environment Variables:** The `JAVA_HOME` environment variable must be explicitly set to point to the JDK installation directory (e.g., `C:\Program Files\Java\jdk-17`). The SuiteCloud extension checks this variable upon startup; if it is missing or points to a JRE (Java Runtime Environment) instead of a JDK, the extension features will remain disabled.

1.1.2 Node.js Runtime Requirements

While Java handles the backend communication, the modern SuiteCloud CLI (specifically the Node.js version) utilizes the Node.js runtime for asynchronous file handling, project scaffolding, and package management.

- **Supported Versions:** The officially supported version is **Node.js version 22 LTS**.¹ Developers using older LTS versions like Node 14 or 16 often encounter compatibility issues with the latest SuiteCloud npm packages.
- **Role in the Ecosystem:** Node.js powers the client-side validation logic. When a developer runs a command to create a project or validate a script, Node.js processes the local file system, calculates checksums, and manages the JSON configuration files

(project.json, suitecloud.config.js) before handing off the payload to the Java layer for transmission.

1.2 Visual Studio Code and the SuiteCloud Extension

Visual Studio Code (VS Code) has displaced Eclipse as the industry-standard IDE for NetSuite development due to its lightweight architecture and rich ecosystem of extensions. The **SuiteCloud Extension for Visual Studio Code** transforms the generic text editor into a context-aware NetSuite IDE.

1.2.1 Installation and Initialization

The installation process involves retrieving the extension from the Visual Studio Marketplace and ensuring the initialization of the underlying language server.

1. **Acquisition:** The extension is published by Oracle and can be found by searching for "SuiteCloud Extension for Visual Studio Code" in the Extensions view (Ctrl+Shift+X).³
2. **Post-Install Initialization:** Upon installation, a restart of VS Code is mandatory. During the first launch, the extension attempts to detect the JAVA_HOME path and the SuiteCloud CLI installation. If the CLI is not found locally, the extension may attempt to download the necessary binaries or prompt the user to install the @oracle/suitecloud-cli package globally via npm.
3. **Verification:** Success is verified by opening the Command Palette (Ctrl+Shift+P on Windows/Linux, Cmd+Shift+P on macOS) and typing SuiteCloud. A list of commands such as SuiteCloud: Create Project and SuiteCloud: Set Up Account should appear, indicating the extension is active and loaded.³

1.2.2 The Command Palette Workflow

The design philosophy of the SuiteCloud extension prioritizes a keyboard-centric workflow. Rather than navigating complex menu trees, developers interact with the environment almost exclusively through the Command Palette. This reduces context switching and aligns with the workflow of modern JavaScript and TypeScript developers. All critical lifecycle events—creating projects, deploying code, importing objects, and comparing files—are triggered via SuiteCloud: commands.³

2. Authentication Architecture: The Transition to OAuth 2.0

The mechanism by which a local development environment authenticates with the NetSuite cloud has undergone a radical transformation. The user query's reference to "token, appid, account id" alludes to the legacy Token-Based Authentication (TBA) model. However, to provide a future-proof guide, this report focuses on the modern **OAuth 2.0** implementation,

which became the mandatory standard starting in 2024 and 2025.

2.1 The Deprecation of Legacy TBA

Historically, developers manually generated a "Token ID" and "Token Secret" in the NetSuite UI and pasted them into a suitecloud.properties file or the IDE setup screen. As of February 2025, new installations of the SuiteCloud Extension (versions 1.6.0 and later) have deprecated support for this manual TBA setup in favor of OAuth 2.0 protocols.⁶ This shift enhances security by removing the need for developers to handle long-lived static secrets and enabling better session management.

2.2 Browser-Based Authentication (Interactive Mode)

For individual developers working on local workstations, **Browser-Based Authentication** is the standard and most streamlined method. It utilizes the OAuth 2.0 Authorization Code Grant flow.

Operational Workflow:

1. **Initiate Setup:** Open the Command Palette and execute SuiteCloud: Set Up Account.
2. **Select Authentication Mode:** Choose **Browser-based authentication** from the dropdown menu.³
3. **Define Authentication ID (AuthID):** The system prompts for an *Authentication ID*. This is a critical concept in SDF. The AuthID is a local alias (e.g., MySandbox_Admin or ClientX_Prod) that represents the stored credentials. It allows developers to switch between accounts (e.g., Sandbox vs. Production) by referencing a readable name rather than re-entering credentials.³
4. **Domain Configuration:**
 - **Standard Accounts:** Press Enter to use the default system domain (system.netsuite.com).
 - **Sandbox/Release Preview:** If connecting to a Sandbox, the developer *must* use the specific account-based domain structure (e.g., 123456-sb1.app.netsuite.com) if prompted, or rely on the system to redirect based on the Account ID.⁸
5. **The OAuth Handshake:** VS Code launches the default web browser. The developer logs into NetSuite and is presented with a consent screen requesting permission for the "SuiteCloud Development Integration" application. Upon approval, NetSuite issues a refresh token to the local VS Code extension, which is stored securely in the OS keychain. No manual copying of tokens is required.⁹

2.3 Machine-to-Machine (M2M) Authentication (CI/CD Mode)

For automated environments, such as Continuous Integration/Continuous Deployment (CI/CD) pipelines (Jenkins, GitHub Actions), or for developers who prefer a headless setup, **Machine-to-Machine (M2M)** authentication is the required method. This utilizes the OAuth 2.0 Client Credentials Grant flow and relies on RSA certificate security rather than user

passwords.

2.3.1 Cryptographic Key Generation

The M2M flow requires a public/private key pair. This must be generated locally using OpenSSL or a similar tool.

- **Private Key:** openssl genrsa -out private-key.pem 2048 (Kept secret on the developer machine or CI server).
- **Public Key:** openssl rsa -in private-key.pem -pubout -out public-key.pem (Uploaded to NetSuite).¹⁰

2.3.2 NetSuite Server-Side Configuration

Before VS Code can connect, the certificate must be associated with a user in NetSuite.

1. Navigate to **Setup > Integration > Manage Authentication > OAuth 2.0 Client Credentials (M2M) Setup**.
2. **Create Mapping:**
 - **Entity:** Select the NetSuite User (Employee) who will act as the authorized agent.
 - **Role:** Select the specific Role (e.g., Administrator, Developer) that the integration will assume.
 - **Application:** Select "SuiteCloud Development Integration".
 - **Certificate:** Upload the public-key.pem file generated in the previous step.
3. **Certificate ID:** Upon saving, NetSuite generates a unique alphanumeric **Certificate ID**. This ID is the "token" equivalent in the M2M flow and must be recorded.¹¹

2.3.3 VS Code Configuration

1. Execute SuiteCloud: Set Up Account.
2. Select **Machine-to-machine authentication**.
3. **Account ID:** Enter the NetSuite Account ID (e.g., 123456 or 123456_SB1).
 - **Note:** The Account ID can be found in **Setup > Company > Company Information** or derived from the URL.¹³
4. **Certificate ID:** Enter the ID generated by NetSuite in the previous step.
5. **Private Key Path:** Provide the absolute local path to the private-key.pem file.
6. **AuthID:** Assign an alias (e.g., CI_Runner_Prod).

Table 1: Comparison of Authentication Methods

Feature	Browser-Based Authentication	Machine-to-Machine (M2M)
Primary Use Case	Interactive Local Development	CI/CD Pipelines, Headless Servers

Security Mechanism	User Login + OAuth Consent	RSA Certificate (Public/Private Key)
Credential Type	Refresh Token (Stored internally)	Certificate ID + Private Key File
Setup Complexity	Low (One-click)	High (Requires OpenSSL & Mapping)
Expiration	Session-based (Refreshable)	Valid until Certificate Expiry (usually 1-2 years)
User Interaction	Required (Browser popup)	None (Automated)

3. Strategic Project Initialization: ACP vs. SuiteApp

Selecting the correct project type is the single most critical architectural decision a developer makes at the onset of a new initiative. SDF supports two distinct project architectures: **Account Customization Projects (ACP)** and **SuiteApp Projects**. These are not merely configuration differences; they represent fundamentally different distribution models and folder structures.

3.1 Account Customization Projects (ACP)

Target Audience: Internal IT Teams, Administrators, Developers managing a single instance.

ACPs are designed for "bespoke" development intended for a specific NetSuite account (e.g., a company's unique Production or Sandbox instance).

- **Characteristics:**
 - **Scope:** Bound to the customization of a specific account.
 - **Deployment:** Overwrites or updates the specific account's configuration directly.
 - **Publisher ID:** Not required.
 - **Folder Structure:** Flexible, but generally mirrors the standard File Cabinet.
 - **Use Case:** Creating a custom workflow for a company's specific sales process, or a script that integrates with their proprietary legacy system.¹⁵

3.2 SuiteApp Projects

Target Audience: Independent Software Vendors (ISVs), Partners, SuiteCloud Developer Network (SDN) members.

SuiteApp projects are self-contained applications intended for commercial distribution via the **SuiteApp Marketplace** or **SuiteApp Control Center**. They are built to be installed in *any* NetSuite account, adhering to strict isolation rules to prevent conflicts.

- **Characteristics:**
 - **Scope:** Distributable package (Bundle/App).
 - **Identity:** Requires a valid SDN **Publisher ID** (e.g., com.mycompany) and **App ID** (e.g., com.mycompany.taxconnector).
 - **Folder Structure:** Enforces a strict hierarchy (FileCabinet/SuiteApps/com.publisher.app/) to ensure namespace isolation.
 - **Validation:** stricter validation rules apply. For example, circular dependencies are forbidden, and references to standard objects must be handled carefully to ensure they exist in the target account.¹⁵

3.3 VS Code Creation Workflow

The project creation process is wizard-driven within VS Code but requires specific inputs based on the project type.

Step-by-Step Creation:

1. **Initiate:** execute SuiteCloud: Create Project.¹⁸
2. **Select Type:** Choose *Account Customization Project* or *SuiteApp Project*.
3. **Project Name:** Enter the internal project name (e.g., ERP_Optimization_2025).
4. **Publisher ID** (SuiteApp Only): If SuiteApp was selected, you must enter the Publisher ID assigned by NetSuite SDN (e.g., com.acme). If you do not have one, you cannot validly deploy a SuiteApp project to the Marketplace.
5. **Project ID** (SuiteApp Only): The suffix for the App ID (e.g., financeapp). The full App ID becomes com.acme.financeapp.
6. **Project Version:** Standard semantic versioning (e.g., 1.0.0).
7. **Unit Testing:** The wizard asks: "Do you want to include unit testing?" Selecting 'Yes' scaffolds a `__tests__` directory and installs the **Jest** testing framework configuration. This is highly recommended for modern development to enable Test-Driven Development (TDD).¹⁹

Table 2: Structural Differences between ACP and SuiteApp

Component	Account Customization Project (ACP)	SuiteApp Project
Root XML	manifest.xml (Type: ACCOUNTCUSTOMIZATION)	manifest.xml (Type: SUITEAPP)

	)	
File Location	FileCabinet/SuiteScripts/ (Standard)	FileCabinet/SuiteApps/<AppID>/ (Mandated)
Object Prefix	Flexible (e.g., customrecord_)	Strict (e.g., customrecord_appprefix_)
Distribution	Direct Deploy to Account	Marketplace / Control Center
Installation Scripts	Not Supported directly in deploy.xml ²⁰	Supported (Run beforeInstall, afterInstall)
Locking	Objects are generally editable	Objects can be locked via locking.xml

4. The Anatomy of a SuiteCloud Project: Metadata and XML

Understanding the file structure is non-negotiable for SDF mastery. A SuiteCloud project is a collection of XML files that represent NetSuite objects (metadata) and JavaScript files that represent logic.

4.1 manifest.xml: The Project Identity

The manifest.xml file, located at the project root, defines the project's metadata and environment requirements.

- **Dependencies:** If a script utilizes a specific NetSuite feature (e.g., "Serialized Inventory" or "Multi-Subsidiary"), this feature *must* be declared in the manifest. If the project is deployed to an account where this feature is disabled, SDF will block the deployment to prevent runtime failures.

XML

```
<manifest projecttype="ACCOUNTCUSTOMIZATION">
  <projectname>Finance Customizations</projectname>
  <dependencies>
    <features>
      <feature required="true">SERIALIZEDINVENTORY</feature>
      <feature required="true">SERVERSIDESCRIPING</feature>
```

```
</features>
</dependencies>
</manifest>
```

Insight: The SuiteCloud: Add Dependency References to the Manifest command in VS Code scans the project's objects and automatically populates this section, reducing manual errors.²¹

4.2 deploy.xml: The Deployment Map

The deploy.xml file controls the scope of what gets transmitted to NetSuite. It allows developers to maintain files in their local workspace (e.g., documentation, design assets, .git folders) that are *not* uploaded to the NetSuite File Cabinet.

- **Wildcards:** The path ~/FileCabinet/SuiteScripts/* tells SDF to upload all files in that directory recursively.
- **Granularity:** Developers can list specific files to deploy only a subset of the project. This is a common technique to speed up development loops—deploying a single script file takes seconds, whereas deploying a full project with hundreds of objects can take minutes.

XML

```
<deploy>
  <files>
    <path>~/FileCabinet/SuiteScripts/MyScript.js</path>
  </files>
  <objects>
    <path>~/Objects/customscript_my_script.xml</path>
  </objects>
</deploy>
```

Note: The tilde (~) represents the project root directory.²²

4.3 suitecloud.config.js: The Build Pipeline Hook

This JavaScript file (introduced with the Node.js CLI) allows for the customization of the build and deployment pipeline. It exports a configuration object where developers can hook into events like onCompleted (to run a script after deployment) or customize the bundling logic. It serves as the bridge between standard CLI commands and custom DevOps logic, allowing for the injection of environment-specific variables or the execution of pre-deployment test suites.²⁴

5. SuiteScript 2.x Development in SDF

Developing SuiteScript in SDF differs fundamentally from the UI-based approach. In the UI, a developer uploads a .js file and then clicks "Create Script Record." In SDF, both the file and the record are code artifacts.

5.1 The Separation of Concerns: Code vs. Metadata

To deploy a functional User Event script, the project must contain three distinct elements, often spread across two files:

1. **The Logic:** A JavaScript file (e.g., UserEvent.js) containing the SuiteScript 2.x code.
2. **The Script Record:** An XML object defining the script type, name, and linking to the JS file.
3. **The Deployment Record:** An XML definitions defining *where* the script runs (e.g., on the Sales Order record).

5.2 Creating the XML Wrapper

The XML wrapper acts as the configuration layer for the script.

Step-by-Step Implementation:

1. **Create the JS File:** Place the logic file in src/FileCabinet/SuiteScripts/.
2. **Create the Object File:** Create an XML file in src/Objects/ (e.g., customscript_ue_sales_validation.xml).
3. **Define the Metadata:**

The following XML structure demonstrates a complete User Event script definition. Note how the scriptfile tag references the file path, and the scriptdeployment tag is nested within the script object.

XML

```
<usereventscript scriptid="customscript_ue_sales_validation">
  <name>Sales Order Validation UE</name>
  <description>Prevents saving orders under specific margin conditions.</description>
  <isinactive>F</isinactive>
  <notifyowner>T</notifyowner>
  <scriptfile></scriptfile>
  <scriptdeployments>
    <scriptdeployment scriptid="customdeploy_ue_sales_val_so">
      <recordtype>SALESORDER</recordtype>
      <status>TESTING</status>
      <loglevel>DEBUG</loglevel>
      <executioncontext>USERINTERFACE</executioncontext>
      <allroles>T</allroles>
    </scriptdeployment>
  </scriptdeployments>
</usereventscript>
```

Critical Analysis of the XML:

- **scriptid:** This must be unique across the entire account. For SuiteApps, it usually includes the bundle prefix.
- **scriptfile:** The syntax `` is a dynamic reference. SDF resolves this path at build time. If the file does not exist at that path in the project, validation will fail.
- **status:** Setting this to TESTING ensures the script only runs for the script owner, a crucial safety feature during development.²¹

5.3 Account-Specific Values (ASV)

A common challenge in SDF is the handling of **Account-Specific Values**. For example, if a script deployment references a specific Role ID (e.g., customrole123) or a specific File ID (e.g., 105) that exists in the Sandbox but has a different ID in Production, the deployment will fail or reference the wrong object.

- **Mitigation Strategy:**
 - **Abstraction:** Avoid hardcoding integer IDs. Use Script Parameters (custscript_param_role_id) to inject IDs at runtime.
 - **Warning vs. Error:** In suitecloud.config.js or via CLI flags, developers can set the validation mode for ASVs to WARNING. This allows the deployment to proceed even if an ID is suspect, though it transfers the risk to the runtime execution.²⁶

6. Deployment Strategy: Validation to Production

Deploying code to a NetSuite account via VS Code is a destructive operation—it overwrites the state of the target account with the state of the local XML. Therefore, a disciplined deployment strategy is essential.

6.1 Managing Target Accounts

The "Target Account" is not a static line in a configuration file but a dynamic state of the IDE. This flexibility allows a single project to be deployed to multiple environments (Dev -> Sandbox -> Production) without code changes.

Switching Contexts:

1. **Visual Check:** The VS Code status bar typically displays the currently active AuthID (e.g., Sandbox1).
2. **Changing Targets:**
 - Open Command Palette: SuiteCloud: Set Up Account.
 - Select the AuthID of the desired target.
 - This action updates the project.json or user preferences to point the CLI to the new

environment. All subsequent deploy commands will target this account.²⁷

6.2 The Deployment Lifecycle

The command SuiteCloud: Deploy Project initiates a complex sequence of operations²⁸:

1. **Local Validation:** The CLI checks XML syntax and file structure locally using Node.js.
2. **Server-Side Validation:** The project is zipped and transmitted to NetSuite's "Pre-Install" validator.
 - o NetSuite checks for circular dependencies.
 - o It verifies that all referenced features are enabled in the target account.
 - o It checks for "Locking" conflicts (e.g., trying to modify a record locked by another Bundle).
3. **Deployment (The Write Operation):**
 - o If validation passes, the files are written to the File Cabinet.
 - o The XML objects are parsed, and the metadata tables in the database are updated.
 - o **Audit Trail:** NetSuite creates a "Deployment Audit Trail" record, logging who deployed what and when.³⁰
4. **Post-Deployment Hooks:** If configured in suitecloud.config.js, automated tests or notification scripts are executed.

6.3 Smart Upload vs. Full Project Deploy

For rapid iteration loops, deploying the entire project (which includes zipping and validating all objects) is inefficient.

- **Smart Upload:** The extension allows developers to right-click a specific .js file and select **Upload File**.
- **Limit:** This *only* updates the file content. It does *not* update metadata. If the developer added a new parameter to the XML script record, a "Smart Upload" of the JS file will not reflect this change. A full SuiteCloud: Deploy Project is required for metadata changes.²⁸

7. Advanced Topics: CI/CD and Team Workflows

The ultimate goal of using SDF is to integrate NetSuite development into standard DevOps pipelines.

7.1 Continuous Integration (CI)

By using the **Machine-to-Machine (M2M)** authentication configured earlier, developers can set up pipelines in GitHub Actions, Azure DevOps, or Jenkins.

Typical CI Workflow:

1. **Commit:** Developer commits code to the develop branch.

2. **Trigger:** GitHub Action triggers a build.
3. **Authenticate:** The Action uses the stored Private Key (as a GitHub Secret) and the suitecloud account:setup:ci command to authenticate.
4. **Validate:** The Action runs suitecloud project:validate. This ensures that the committed code is syntactically correct and compatible with the target NetSuite instance *before* it is merged or deployed.
5. **Test:** The Action runs npm test to execute Jest unit tests.

7.2 Unit Testing with Jest

The SuiteCloud SDK includes a specialized adapter for **Jest**, the popular JavaScript testing framework.

- **Mocking:** The framework allows developers to "mock" NetSuite modules (N/record, N/search). This means unit tests can run locally on the build server without needing to connect to a real NetSuite account.
- **Speed:** Because these tests run in Node.js locally, they execute in milliseconds, providing immediate feedback on logic errors.¹⁹

8. Conclusion

The transition from SuiteBundler and UI-based editing to the SuiteCloud Development Framework represents a maturation of the NetSuite platform. By adopting VS Code, OAuth 2.0, and file-based project architecture, organizations gain the ability to apply rigorous software engineering standards—version control, peer review, and automated testing—to their ERP customizations.

Success in this ecosystem requires a shift in mindset: the "Source of Truth" moves from the NetSuite account to the Git repository. The NetSuite account becomes merely a runtime environment, ephemeral and reproducible from the source code. While the learning curve for configuring JDKs, Certificates, and XML metadata is steeper than the drag-and-drop past, the resulting stability, security, and scalability of the solution provide a definitive competitive advantage for enterprise development teams.

Works cited

1. SuiteCloud CLI for Node.js Installation Prerequisites - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_1558708810.html
2. NetSuite Applications Suite - Installation Prerequisites for SuiteCloud Extension for Visual Studio Code - Oracle Help Center, accessed February 9, 2026, https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_159223197655.html
3. SuiteCloud Extension for Visual Studio Code, accessed February 9, 2026,

<https://marketplace.visualstudio.com/items?itemName=Oracle.suitecloud-vscode-extension>

4. SuiteCloud Development Framework (SDF) Guide for SC - Unlock Commerce, accessed February 9, 2026,
<https://unlockcommerce.co/suitecloud-development-framework-guide/>
5. Installing and Setting Up SuiteCloud Extension for Visual Studio Code - Oracle Help Center, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/article_159223155621.html
6. NetSuite Applications Suite - SuiteCloud CLI for Node.js Guide - Oracle Help Center, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/chapter_1558708800.html
7. NetSuite Applications Suite - SuiteCloud Extension for Visual Studio Code Overview, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/chapter_158400948906.html
8. Finding Your NetSuite Account ID - Oracle Help Center, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_1498754928.html
9. NetSuite Applications Suite - Adding an Account in SuiteCloud ..., accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/subsect_160147727190.html
10. NetSuite Applications Suite - OAuth 2.0 Client Credentials Setup - Oracle Help Center, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_162686838198.html
11. NetSuite Applications Suite - OAuth 2.0 Authentication for ..., accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/article_0422011927.html
12. NetSuite Applications Suite - Setting Up Token-based Authentication - Oracle Help Center, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_161942084079.html
13. How To Find Your NetSuite Account ID - Netwrix, accessed February 9, 2026,
<https://netwrix.com/en/resources/blog/how-to-find-your-netsuite-account-id/>
14. How to Find Your NetSuite Account Number or ID | Gurus Solutions, accessed February 9, 2026, <https://gurussolutions.com/blog/find-netsuite-account-number>
15. Converting Account Customization to SuiteApp Project - NetSuite Folio3, accessed February 9, 2026,
<https://netsuite.folio3.com/blog/converting-account-customization-to-suiteapp-project/>

16. NetSuite Applications Suite - SuiteCloud Project Types - Oracle Help Center, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_4706651345.html
17. SuiteCloud Project Types - GitLab, accessed February 9, 2026,
https://netsuitedocumentation1.gitlab.io/netsuitedocumentation1/section_4706651345.html
18. NetSuite Applications Suite - Creating a SuiteCloud Project in SuiteCloud Extension for Visual Studio Code - Oracle Help Center, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_162938506015.html
19. SuiteCloud Development Framework: Environment setup, commands, and SuiteApp deployment guide - Rand Group, accessed February 9, 2026,
<https://www.randgroup.com/insights/oracle-netsuite/suitecloud-development-framework-environment-setup-commands-and-suiteapp-deployment-guide/>
20. NetSuite Applications Suite - SDF Installation Scripts as XML Definitions, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_1547147541.html
21. NetSuite Applications Suite - Create a Script Record and Script ..., accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_4698045140.html
22. NetSuite Applications Suite - SuiteCloud Project Deployment Preparation, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_4737888643.html
23. Preparing a SuiteCloud Project for Deployment - GitLab, accessed February 9, 2026,
https://netsuitedocumentation1.gitlab.io/netsuitedocumentation1/section_4737888643.html
24. NetSuite Applications Suite - About the Project Structure in ..., accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_155931221634.html
25. Exploring SuiteCloud Development Framework (SDF) by Oracle - Unlock Commerce, accessed February 9, 2026,
<https://unlockcommerce.co/exploring-suitecloud-development-framework-sdf-by-oracle/>
26. I m reaching out to gather insights on managing workflow dep NetSuite Professionals #sdf, accessed February 9, 2026,
<https://archive.netsuiteprofessionals.com/t/16034869/i-m-reaching-out-to-gather-insights-on-managing-workflow-dep>
27. Setting Up NetSuite Accounts in SuiteCloud Extension for Visual Studio Code,

- accessed February 9, 2026,
<https://support.jcurvesolutions.com/blog/setting-up-netsuite-accounts-in-suitecloud-extension-for-visual-studio-code>
28. Validate and Deploy the SuiteCloud Project to a Target NetSuite Account, accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_4698045435.html
29. accessed February 9, 2026,
https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_160147342366.html#:~:text=To%20deploy%20a%20SuiteCloud%20project,xml%20%2C%20locking.
30. SuiteCloud Development Framework: Building and Deploying Customization - NetSuite, accessed February 9, 2026,
<https://nlcorp.app.netsuite.com/core/media/media.nl?id=168412566&c=NLCORP&h=7305d8367607d4092d8e&xt=.pdf>
31. New extension for VS Code - SuiteCloud Smart Upload : r/Netsuite - Reddit, accessed February 9, 2026,
https://www.reddit.com/r/Netsuite/comments/1qbkt9v/new_extension_for_vs_code_suitecloud_smart_upload/
32. NetSuite SuiteCloud Platform Development, accessed February 9, 2026,
<https://www.netsuite.com/portal/platform/developer/suitescript.shtml>